ALBERTO

Developer Guide

Website structure, styling, scripts, assets, and maintenance

Project: Alberto Watch Company
Prepared from the supplied HTML files
Home page • Collections • FAQ section

1. Purpose and Scope

This guide explains how to understand, run, update, and troubleshoot the Alberto Watch Company website using the three supplied files. It is written for a developer maintaining the current HTML/CSS/JavaScript implementation.

- alberto_home_with_footer_ticker_visitor_count(4).html — main landing page, navigation, brand sections, FAQ content, contact area, footer, clock/location ticker, and browser-local visitor count.
- collections(1).html — watch collection catalogue, search/filter interface, product cards, cart interactions, checkout-related UI, and product imagery.
- FAQ_Section.html — standalone FAQ section with a Bootstrap accordion and its own responsive styling.

The supplied project is primarily a front-end website. The reviewed files do not demonstrate a connected server, database, payment gateway, or production authentication service.

2. Technology Stack

- HTML5: page structure, navigation, sections, product cards, forms, and footer.
- CSS3: custom theme, responsive breakpoints, layout, hover states, backgrounds, and typography.
- JavaScript: clock/location updates, visitor counter, collection filtering, cart state, and UI interactions.
- Bootstrap 5: responsive layout utilities, navbar behavior, modal/accordion components, and bundled JavaScript.
- Font Awesome: icons used in navigation, contact details, and controls.
- Swiper: product/trending slider on the main page.
- AOS (Animate On Scroll): scroll-triggered reveal animations.
- Lightbox2: enlarged image viewing where configured.
- SweetAlert2: alert/confirmation UI referenced by the collections page.

3. Project Folder Setup

Keep the HTML files and referenced assets in a predictable folder structure. The exact image inventory should be checked against the source folder before deployment.

```
Alberto-Website/
├── index.html
├── collections (1).html
├── FAQ_Section.html
├── logo.png                  (referenced by collections page)
├── product/
│   ├── bg.png
│   ├── rolex1.jpg ...
│   ├── omega1.jpg ...
│   ├── casio1.jpg ...
│   └── g-shock1.jpg ...
└── aboutpic/
    └── rolexomega.jpg
```

Important: filenames and paths are case-sensitive on many web servers. If an image does not load, verify the actual filename and the relative path from the HTML file.

4. Running the Website Locally

1. Create a project folder and place the three HTML files inside it.
2. Copy the required image folders and logo into the expected relative paths.
3. Open the main HTML file in Chrome, Edge, or Firefox. For more reliable local development, use VS Code with a local preview server such as Live Server.
4. Keep an internet connection available because Bootstrap, Font Awesome, Swiper, AOS, Lightbox2, Google Fonts, and SweetAlert2 are loaded from CDNs.
5. Test each page at desktop, tablet, and mobile widths and verify navigation links and image paths.

The main page and collection page currently have long filenames. For easier maintenance, consider renaming them to index.html and collections.html, then update every link that refers to the old names.

5. Main Page: Key Areas

- Navigation bar: primary site navigation, dropdown links, search/cart entry points, and mobile collapse behavior.
- Hero area: large promotional introduction with responsive typography and background/image treatment.
- Gallery and brand sections: content areas with IDs such as home, gall, rolex-related section, omega-related section, casio-related section, and g-shock-related section. Check the actual anchor targets before changing navigation URLs.
- About section: company introduction and brand/service information.
- FAQ section: FAQ accordion markup is present in the supplied main page.
- Contact area: contact details and an inquiry form. The current form uses a mailto action, which relies on the visitor having a configured email application.
- Footer: company links, social/payment or trust visuals, and current-year output.
- Ticker utilities: JavaScript functions updateClock() and setLocation() are present for date/time and location display.
- Visitor counter: uses browser localStorage in the current implementation. It counts visits in that browser profile, not a global total across all visitors.

6. Collections Page: Product and Cart Logic

The collections page contains product cards for Rolex, Omega, G-Shock, and Casio models. Product image paths follow a product/filename pattern, so keep image names consistent with the HTML.

- filterWatches(): filters or narrows the visible watch results based on the search/filter input.
- addToCart(productName, price): updates the in-memory cart items and cart count.
- removeFromCart(...): removes an item from the cart state where supported by the implementation.
- showCartItems(): displays the current cart UI/modal.
- proceedToCheckout(): handles the current checkout flow in the front end; review the implementation before treating it as a real payment process.
- sessionStorage checkout/login checks: the page reads sessionStorage keys such as checkoutPending and loggedIn to control parts of the demo flow.

Developer note: a cart stored only in JavaScript memory resets when the page reloads. sessionStorage is temporary and tab/session-scoped. A real multi-user store needs a backend, persistent product/order storage, server-side validation, and a payment provider.

7. FAQ Section

- FAQ_Section.html is a standalone FAQ section with the page title “FAQs | Alberto Watch Company”.
- The accordion uses Bootstrap data attributes such as data-bs-toggle="collapse" and data-bs-target to open and close answers.
- The Bootstrap bundle script at the end of the document is required for the accordion interactions.
- CSS custom properties in :root control the FAQ palette, including background, card, text, muted text, gold, border, and dark colors.
- The current FAQ includes nine questions covering site exploration, brands, product details, purchasing, login behavior, repair/appraisal services, opening hours, location, and contact.

To integrate the FAQ section into another page, copy the section markup and its required CSS, ensure IDs are unique, and load the Bootstrap bundle only once per page. Avoid nesting a full HTML document inside another document.

8. Styling and Responsive Design

The website uses inline style blocks in each HTML file. CSS classes control navigation, hero typography, cards, slider controls, modal appearance, footer columns, and responsive behavior.

- Use CSS custom properties for repeated brand colors so the theme can be changed centrally.
- The collections page uses a wine/burgundy, champagne-gold, cream, and dark palette.
- Media queries adjust the slider, navbar, and footer layout at smaller screen widths.
- Use browser DevTools → Inspect to identify the active selector when a style does not apply.
- Avoid copying a stylesheet link to an HTML document. A link rel="stylesheet" must point to a CSS file, not another .html page. The supplied collections file includes a stylesheet link whose href points to an HTML file; this should be corrected by extracting shared CSS into a .css file or removing the incorrect link.

9. External Dependencies

CDN dependencies are convenient for an assignment/demo but rely on internet access and third-party availability. Keep each dependency version consistent and avoid loading the same library multiple times.

- Bootstrap CSS and bootstrap.bundle.min.js
- Font Awesome CSS
- Swiper CSS/JavaScript
- AOS CSS/JavaScript
- Lightbox2 CSS/JavaScript
- Google Fonts on the FAQ page
- SweetAlert2 on the collections page

Before production, verify that the CDN URLs and integrity hashes match the versions in use. Do not remove a library until its corresponding components and scripts have been tested.

10. Known Limitations and Checks

- No server-side application or database is shown in the supplied HTML files.
- The visitor count is browser-local; clearing site data or changing browsers can reset it.

- The contact form uses mailto and may open the user's default email application rather than submit to a website server.
- Cart and checkout behavior appears front-end/demo-oriented; do not claim that real orders or payments are processed without backend integration.
- A login/session flag is not equivalent to secure user authentication.
- Geolocation requires user permission and may require HTTPS or localhost depending on browser behavior.
- Check for missing image assets and broken navigation targets after moving or renaming files.
- The collections page includes a link element targeting an HTML page as if it were a CSS stylesheet; shared styles should be placed in a real .css file.

11. Recommended Maintenance Workflow

6. Make a backup or commit changes to version control before editing.
7. Update one area at a time: structure first, styles second, scripts third.
8. Keep product image names and paths aligned with the product cards.
9. Test search, add/remove cart, cart modal, FAQ accordion, navbar collapse, ticker, and contact link after edits.
10. Use DevTools Console and Network tabs to inspect JavaScript errors and missing assets.
11. Check keyboard navigation, readable contrast, image alt text, and responsive layout.
12. If moving beyond a demo, separate HTML, CSS, and JavaScript into dedicated files and introduce a backend for authentication, inventory, orders, and payments.

12. Quick Troubleshooting

Problem	What to check
CSS not applying	Check selector specificity, spelling, closing braces, stylesheet path, and whether a link points to a real .css file.
Product image missing	Check relative path, exact case, file extension, and whether the image exists in the product folder.
Accordion does not open	Confirm Bootstrap bundle is loaded and data-bs-target matches the answer element ID.
Search returns no results	Check input IDs, filterWatches() event wiring, and product text/attributes used for matching.
Cart count is wrong	Inspect addToCart/removeFromCart logic and confirm the cart state is not being reset unexpectedly.
Location is blank	Check browser permission, secure-context requirements, and error handling in setLocation().
Visitor count looks different on another device	Expected with localStorage; a shared count requires a server-side database.

13. Suggested Future Improvements

- Move repeated navigation/footer CSS into a shared stylesheet and shared markup component/template.
- Rename files to simple stable names and update links consistently.
- Store product data in a structured JSON file or backend instead of duplicating card markup.
- Add automated checks for broken links, missing images, and JavaScript errors.
- Use a secure backend for real accounts, stock, orders, and payment processing.
- Add accessible labels and meaningful alt text to all interactive controls and product images.

Appendix: Important Function Names

```
Main page: updateClock(), setLocation(), showCartItems()
Collections: addToCart(), removeFromCart(), showCartItems(), proceedToCheckout(),
filterWatches()
FAQ: Bootstrap accordion controlled by data-bs-toggle="collapse"
```

This document describes the supplied files as reviewed. If the code changes, update this guide so it continues to match the running website.

> *Developer Note: This functional flowchart blueprint serves as an operational map detailing the relationship between pages, state flags, tracking metrics, client-side behaviors, and troubleshooting patterns specified in the Developer Guide.*

1. Core System Component Flowcharts

The functional modules of the website are structurally organized below into linear procedural workflows mapping user interaction vectors, initialization loops, local storage mutations, and frontend logic cascades.

Flowchart Group A: Main Landing Page (index.html)

Step / Node	System Entity / Action	Functional Description & Technical Notes
A1: Entrance	User Accesses Landing Page	Triggers initialization scripts. Loads layouts, sliders, and background layers.
A2: Tracker Loop	Local Storage Visitor Count Check	Reads ' visitor-count ' key from browser. Increments and writes local profile state.
A3: UI Utilities	Update Clock() & set Location()	Polled utilities render regional timestamp and execute browser Geolocation API loops.
A4: Scroll Hooks	AOS / Swiper.js Initialization	Triggers viewport boundary evaluation to fire scroll-driven reveal styles and animations.
A5: Widget Logic	Bootstrap FAQ Component	Evaluates 'data-bs-toggle=collapse' nodes on layout targets to trigger accordion states.

A6: Form Submit	Contact Form mail to Directive	Constructs standard mail to schema string and invokes local default mail client runner.

Flowchart Group B: Watch Collections Catalogue (collections.html)

Step / Node	State Matrix / Operation	Functional Description & Technical Notes
B1: Catalog View	Load Product Matrix Grid	Renders dataset card markup for watch lines (Rolex, Omega, Casio, G-Shock).
B2: Search Hook	filterWatches () Process	Listens for text matching against descriptive element tokens to filter grid elements.
B3: State Event	addToCart (product, price)	Appends product reference record to volatile memory array. Updates layout count badge.
B4: Modal View	showCartItems() Overlay	Constructs transient listing from state memory array and renders checkout path hooks.
B5: State Splice	Remove From Cart() Action	Locates element key index inside state array, cuts target entry, and re-renders modal UI.
B6: Checkout Validation	Proceed To Checkout() Hook	Checks sessionStorage keys ('loggedIn', 'checkoutPending') to handle checkout gateway demo flow.

Flowchart Group C: Maintenance & Debugging Engines

Error State Node	Target Validation Hook	Resolution Path & Code Correction Patch

C1: Broken Styling	Stylesheets File Integrity	Locate link elements pointing incorrectly to external HTML files; swap for raw CSS files.
C2: Asset Drops	Image File Paths & Extensions	Verify case-sensitive absolute matching paths targeting resources located within /product.
C3: Broken Accordion	Bootstrap Component Bindings	Ensure data-bs-target values exactly reflect valid target ID values of answer blocks.
C4: Cart Clearance	Array State Scope Lifecycle	Ensure state persistence strategies match limits; standard memory arrays reset on reload.

2. Cross-Page Interconnection Mapping

The visual routing and operational links running across individual isolated frontend environments map as follows:

Navbar Link Vectors: Provide bi-directional connection between index.html and collections.html. Maintenance guide strongly advises renaming files to permanent short patterns to prevent path decay.

Isolated FAQ Component Integration: FAQ_Section.html houses a distinct standalone Bootstrap context. To safely blend this component with index.html, target blocks must be moved without wrapping secondary HTML shells inside existing document trees.